Setting up your computer

NSDF-Data student guide 1 — Linux edition

Version 23 · 5 November 2026

Tip

Who this is for. You are starting from little or no computing experience. You will paste some commands into a terminal and read the answers. You do **not** need to know any programming. Work through the steps in order. Each step ends with a **Checkpoint**: if it does not match, stop there.

Before you start

- **Time:** about one hour. Most of it is waiting for downloads and installers.
- **You need:** an internet connection, the password for your computer, and about 3 GB of free disk space.
- **Rule of thumb:** type or paste commands **exactly** as written. Capitals and spaces matter.
- **What to type, and what not to:** you type only the commands: text in a shaded box, or shaded inline like `git --version`. A checkpoint that says *You should see* describes what the computer prints back: do not type that. The symbol at the start of the line in the terminal (`$`, `>` or `%`) is the prompt the terminal shows you: do not type that either.
- **Nothing to be afraid of:** the commands in this guide install software and make new folders. None of them deletes your files. If you are not sure what a line does, there is a short table after each group of commands.
- **When something goes wrong:** do not guess. Copy the message, note the step you were on, and ask the project lead. See *If something goes wrong* at the end.

Words you will meet

Word	What it means
Terminal	A window where you type commands and the computer answers in text.
Command	One line you type in the terminal, followed by **Enter** (or **Return**).
Folder (or directory)	A place on your computer that holds files and other folders.
Path	The address of a folder, such as `C:\Users\YourName\Research`.

Word	What it means
Git	A program that downloads and keeps track of versions of code.
Repository (repo)	A project folder that Git tracks. Here, `NSDF-Data`.
Python	The programming language the analysis is written in. You will not need to write Python to finish these guides.
Conda / Miniforge	A tool that installs Python and packages in separate, named *environments*.
Environment	A sealed set of programs and versions, so this project's software does not clash with others.
Package	A piece of ready-made software (for example `numpy`).
Jupyter notebook	A document that mixes text, code and plots, run one piece (*cell*) at a time.
Branch	A separate line of work in Git, so your changes do not touch the main code.

Step 1. Open a terminal

Press **Ctrl + Alt + T** (on Ubuntu), or search your applications for **Terminal**. A window with a text prompt opens. This is the terminal for everything in this guide.

Paste with **Ctrl + Shift + V** (plain Ctrl+V does not work in most terminals).

Checkpoint

You see a line of text ending in a symbol such as >, $ or %. Type `echo hello` and press Enter: the word `hello` should appear under it.

Step 2. Install Git

First check whether Git is already installed. Type:

```
git --version
```

If you see a line starting with `git version`, Git is installed: skip the rest of this step and go to Step 3. If the terminal says the command is not found, install it. This guide uses Ubuntu or Debian commands (on another distribution, use its package manager instead). Type:

```
sudo apt install git
```

It asks for your password. Nothing appears while you type it; that is normal. Press **Return**, then type `y` and press **Return** if it asks to continue.

If you cannot use `sudo` (a computer you do not administer, such as a shared or university machine, or `sudo` says you are not allowed to): skip the `apt` command. Do Step 3 first. Then, in the new terminal, run `conda install -y -c conda-forge git`. That installs Git in your own folder

and needs no password. Come back to the checkpoint below and run `git --version`.

Checkpoint

In the terminal, type `git --version` and press Enter. You should see a line starting with `git version`.

Step 3. Install Miniforge

Miniforge installs Python and the other packages the analysis needs. Its download file is **Miniforge3-Linux-x86_64.sh (most PCs)**.

Check your computer type with this command, which prints one word:

```
uname -m
```

`x86_64` means the file below. `aarch64` means an ARM computer: use **Miniforge3-Linux-aarch64.sh** instead.

Download the file into your **Downloads** folder: Miniforge3-Linux-x86_64.sh, or Miniforge3-Linux-aarch64.sh for an ARM computer. Then install it with this command (use the file name you downloaded):

```
bash ~/Downloads/Miniforge3-Linux-x86_64.sh
```

The installer asks its questions in the terminal. They read like this (the wording can differ a little between versions):

What the installer shows	What you do
`Please, press ENTER to continue`	Press **Return**. The licence text starts to scroll.
The licence text, then the question `Do you accept the license terms?`	Press **Return** (or **Space**) again and again until the text ends and the question appears. Then type `yes` and press **Return**.
`Miniforge3 will now be installed into this location`, with a folder name	Press **Return** to accept the folder.
`Do you wish to update your shell profile to automatically initialize conda?`	Type `yes` and press **Return**.

Then it prints a lot of text, and ends with `Thank you for installing Miniforge3!`. Now **close the terminal and open a new one**.

Checkpoint

Open a **new** terminal and type `conda --version`, then press Enter. You should see `conda` followed by a version number.

Careful

Use `sudo` only to install Git in Step 2, and only if `git --version` did not already work. Every other command in this guide must not use it. If you cannot use `sudo`, Step 2 gives a route that

does not need it.

Step 4. Download the NSDF-Data code

Your project will live in a folder called `Research`. Create it, go into it, and download the code by running the four commands below. To run a command, type (or paste) the line into the terminal and press Enter. Wait for the prompt to come back, then do the next line:

```
mkdir -p ~/Research
cd ~/Research
git clone https://github.com/villano-lab/NSDF-Data.git
cd ~/Research/NSDF-Data
```

Downloading with `git clone` does not need a GitHub account (saving your work online later does: Step 8). It makes a full copy of the project in a folder named `NSDF-Data`.

What these lines do

Line starts with	What it does
`mkdir`	Makes a new folder called `Research`.
`cd`	Moves you into a folder. `cd` stands for *change directory*, which is the same as opening a folder in a file window.
`git clone`	Downloads a copy of the project from the internet into a new folder called `NSDF-Data`.

Checkpoint

Type `ls` and press Enter. You are now inside the `NSDF-Data` folder (your prompt mentions it), so the list shows the project's own folders and files, such as `python`, `data_analysis`, `docs` and `README.md`.

Step 5. Create your analysis environment

An environment is a sealed box of programs with fixed versions. We make one for this project, called `darkmatter_cli_env`. Type these lines one at a time, pressing Enter after each. The last line takes several minutes and prints a lot of text: wait for the prompt to come back.

```
conda create -n darkmatter_cli_env python=3.12 pip -y
conda activate darkmatter_cli_env
cd ~/Research/NSDF-Data
python -m pip install -r requirements.txt
```

What these lines do

Line	What it does
`conda create`	Makes a new, empty environment called `darkmatter_cli_env`, with Python 3.12 and the package installer `pip` inside it.
`conda activate`	Switches that environment on. While it is on, your prompt starts with `(darkmatter_cli_env)`.
`cd`	Moves you into the `NSDF-Data` folder, where the file `requirements.txt` is.
`pip install -r requirements.txt`	Installs every program on the project's list, at the exact versions in that file: the software that reads the detector data, numbers and plots, the test tool, and Jupyter, the notebook program you will use later.

Tip

`requirements.txt` is a plain text file in the project folder. It is the single list of what the project needs, so when a version changes, only that file changes. You can open it in any text editor to see the list.

Tip

Every time you open a new terminal, run `conda activate darkmatter_cli_env` first. When it is active, the start of each line shows `(darkmatter_cli_env)`.

Checkpoint

The last line of the output is a message, not an error, and the `(darkmatter_cli_env)` label is at the start of your prompt.

Step 6. Check that everything works

First, the environment and the project's library. Run the four commands below, one at a time: type (or paste) each line into the terminal and press Enter, and wait for the prompt to come back before the next one:

```
conda activate darkmatter_cli_env
cd ~/Research/NSDF-Data
python -m check_environment
python -c "import importlib.metadata as m, goldpan; print('goldpan', m.version('gold
```

Checkpoint

The check says `Environment OK`, and the last line prints `goldpan` and a version number. If the check says `WARNING`, it lists what differs: stop and ask the project lead, and do not change any files.

Then, the data tool:

```
nsdf-cli version
```

Checkpoint

It prints `NSDF Dark Matter CLI: 0.6.0` (or a later version).

Finally, the notebook program:

```
jupyter lab --version
```

Checkpoint

It prints a version number, such as `4.4.0`. If it says `jupyter-lab` is not found, check that your prompt starts with `(darkmatter_cli_env)`, go to the `NSDF-Data` folder and run `python -m pip install -r requirements.txt` again.

What these commands do

Line	What it does
`conda activate`	Switches your environment on. Do this in every new terminal.
`cd`	Moves into the `NSDF-Data` folder of the project.
`python -m check_environment`	Runs the project's environment check. It compares your Python and your packages with the project's list (`requirements.txt`) and prints `Environment OK`, or says what differs.
`python -c "..."`	Asks Python to load the project's library (`goldpan`) and print its version. If it answers, the library is installed.
`nsdf-cli version`, `jupyter lab --version`	Each one only asks a program to say which version it is. If it answers, it is installed.

Step 7. Let Jupyter notebooks use your environment

Jupyter is the notebook program you will use. Register your environment with it once:

```
python -m ipykernel install --user --name darkmatter_cli_env
```

Checkpoint

It prints a line that starts with `Installed kernelspec darkmatter_cli_env`.

Step 8. Get permission to save your work

You will not change the main project. You will save your work on your own **branch**, and for that the lead must give you access:

1. Make a free account at github.com if you do not have one.

2. Send your GitHub username to the project lead. They will add you to the project as a collaborator.
3. Once you have accepted the invitation, tell Git your name and email, so your work is labelled as yours. Use your own details:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

4. Sign in to GitHub from the terminal, so that GitHub lets your computer upload your work. The second line installs `gh`, GitHub's own small helper program. The third starts the sign-in:

```
conda activate darkmatter_cli_env
conda install gh -c conda-forge -y
gh auth login
```

`gh auth login` asks a few questions. Move with the arrow keys and press Enter. Choose **GitHub.com**, then **HTTPS**, then answer **Y** to "Authenticate Git with your GitHub credentials?", then choose **Login with a web browser**. It prints a short one-time code such as `ABCD-1234`. Press Enter: your web browser opens. Type the code there and click **Authorize**. Back in the terminal, check it worked:

```
gh auth status
```

Checkpoint

It says `Logged in to github.com account` followed by your GitHub username.

Careful

Never type your GitHub password into the terminal. GitHub no longer accepts it there, and Git answers with `Password authentication is not supported` or `Invalid username or token`. If you ever see one of those messages, run `gh auth login` again.

5. Create your personal branch (your own space to save work, separate from the main project). Use your own name, without spaces, in place of `yourname`:

```
cd ~/Research/NSDF-Data
git checkout -b student-yourname
```

Checkpoint

The terminal shows `Switched to a new branch 'student-yourname'`.

Using a second or a new computer

Your work lives on GitHub, on your own branch, not on one computer. To work on another computer, or on a replacement, you do **not** make another folder under your name. Set the computer up like this:

1. Do Steps 1 to 7 as written.
2. In Step 8, do items 3 and 4 only (tell Git your name and email, and sign in with `gh auth login`). You already have access, so skip items 1 and 2.
3. **Skip item 5.** `git checkout -b` makes a *new* branch, and yours already exists on GitHub. Bring it down instead, with your own name in place of `yourname`:

```
cd ~/Research/NSDF-Data
git fetch origin
```

```
git switch student-yourname
```

Checkpoint

Git says branch 'student-yourname' set up to track 'origin/student-yourname', and `git branch --show-current` prints student-yourname. Your notebooks and your folder under data_analysis/R76/student/ are there, because they came with the branch.

4. The data is **not** stored on GitHub, so download it again on the new computer (first analysis guide, Step 2).

On the computer you are leaving, **commit and push your work first** (Sync Changes, or the steps in the first analysis guide, Step 7). Anything you did not push is only on that computer. If an old computer starts again later, open it and push before you do anything else.

Rules for keeping your work safe

- **Never work on `main` or `develop`.** Those are the shared versions. Work only on your `student-` branch.
- **Never commit data.** The raw files in the `idx` folder are large external data, not part of our data analysis code in the NSDF-Data repository. Download only from your home folder, never from inside NSDF-Data.
- **Do not run** `07221203_2025_dump1_noise.ipynb` unless the project lead says it is fine. It overwrites a shared archive file.
- **Save often,** and commit your work when a piece is finished (see the first analysis guide).

You are done when

- ☐ Your terminal opens and `git --version` prints a version.
- ☐ `conda --version` prints a version, in a new terminal.
- ☐ `ls ~/Research` lists NSDF-Data.
- ☐ `python -m check_environment` says `Environment OK`. Note: first activate the environment with `conda activate darkmatter_cli_env` and go to the NSDF-Data folder with `cd ~/Research/NSDF-Data`, as in Step 6.
- ☐ `nsdf-cli version` prints a version.
- ☐ `jupyter lab --version` prints a version.
- ☐ `jupyter kernelspec list` lists darkmatter_cli_env.
- ☐ `gh auth status` says you are logged in to github.com (after access is granted).
- ☐ `git branch` shows `* student-yourname` (after access is granted).

Next: *First analysis* (the guide for Linux).

Routine maintenance

Your setup is not a one-time job. The project keeps changing: new releases bring new code and guides, and sometimes a new version of Python or of the packages. **Every time the project lead announces a new release, and whenever the environment check warns you, follow Updating your setup.** It takes about 10 minutes.

If something goes wrong

1. Read the **last line** of the message. It usually says what is wrong.
2. Close the terminal, open a new one, and run `conda activate darkmatter_cli_env` again. Many problems are just the wrong environment.
3. Copy the command you ran and the message it printed, and send them to the project lead. A screenshot is also fine.

Where to get help

Anthony Villano, project lead: anthony.villano@ucdenver.edu

When you write, include the step number, the command you typed, and the last few lines the computer printed. A screenshot helps. Nobody will be annoyed by a question about a step that did not work.

Session Info

Guide version 23, 5 November 2026, Linux edition. Written for beginners; please tell the project lead where a step was unclear, so the guide can be fixed.

Key links

- Notes site: https://villano-lab.github.io/NSDF-Data/
- Repository: https://github.com/villano-lab/NSDF-Data
- Miniforge downloads: https://github.com/conda-forge/miniforge
- Git downloads: https://git-scm.com